Meridian Sentinel

A hybrid fraud-detection prototype — what I built, and what actually happened

UNIT

ITW601 — Capstone Project

IMPLEMENTATION REPORT AND EVIDENCE

Rule engine · LSTM inference · Elasticsearch · React console

1. What this project is

Short version: two detectors look at every payment, their scores get blended into one number, and if that number crosses 0.70 the system locks the account by itself.

Meridian Sentinel is a fraud-detection prototype for a fictional Australian bank. The idea is that neither detection approach is good enough alone. Fixed rules are fast and you can explain them to an auditor, but a fraudster who knows the rules can stay under them. A neural network picks up patterns nobody wrote down, but it can't tell you why, which is a problem when you're about to freeze somebody's account. So I ran both and combined them:

```
threat_score = (LSTM × 0.60) + (SIEM rules × 0.40)
```

Anything at 0.70 or above fires an automated playbook — account locked, incident case opened, analyst notified. Anything below is still scored and logged, it just doesn't trigger containment. The 60/40 split gives the model more weight because it's the half that can catch something new, but the rules still get enough weight to drag a score up on their own.

What's actually running

Everything in this report runs in Docker on my machine. Five services: Elasticsearch, Kibana, Logstash, the model server, and a dev container I run the pipeline from. The React dashboard runs separately with `npm run dev`. Every number and screenshot in here came from executing that stack, not from a mock-up.

COMPONENT	WHAT IT DOES	STATUS
SIEM rule engine	5 deterministic rules	Working
LSTM model	Behavioural scoring, ONNX + FastAPI	Working
Hybrid scorer	Blends both, applies threshold	Working
Playbook engine	Incident + notification on FLAGGED	Working
Elasticsearch	Transactions, incidents, audit trail	Working
React dashboard	Live analyst console	Working
Audit trail	8 stages per transaction	Working

2. The model

It's a stacked LSTM: 128 units, dropout 0.30, then 64 units, dropout again, then a single linear output through a sigmoid. Input is a 5×13 tensor — five transactions, thirteen engineered features each. Output is one number between 0 and 1.

Training data is PaySim, a synthetic dataset of ~6.36 million mobile money transactions. I trained in Colab for 35 epochs with a WeightedRandomSampler to deal with the class imbalance, then exported to ONNX and served it with FastAPI.

METRIC	RESULT	TARGET	MET?
Detection accuracy	99.96%	≥ 98.55%	Yes
False positive rate	0.034%	≤ 0.50%	Yes
Recall	93.54%	–	—
Precision	78.02%	–	—
Decision threshold	0.90	–	—

Scaling is part of the model

One design decision is worth explaining because it's easy to get wrong and it's invisible when it is. All thirteen features are MinMax-scaled into [0, 1] before the model sees them, and the scaler that does it has to be the *training* one.

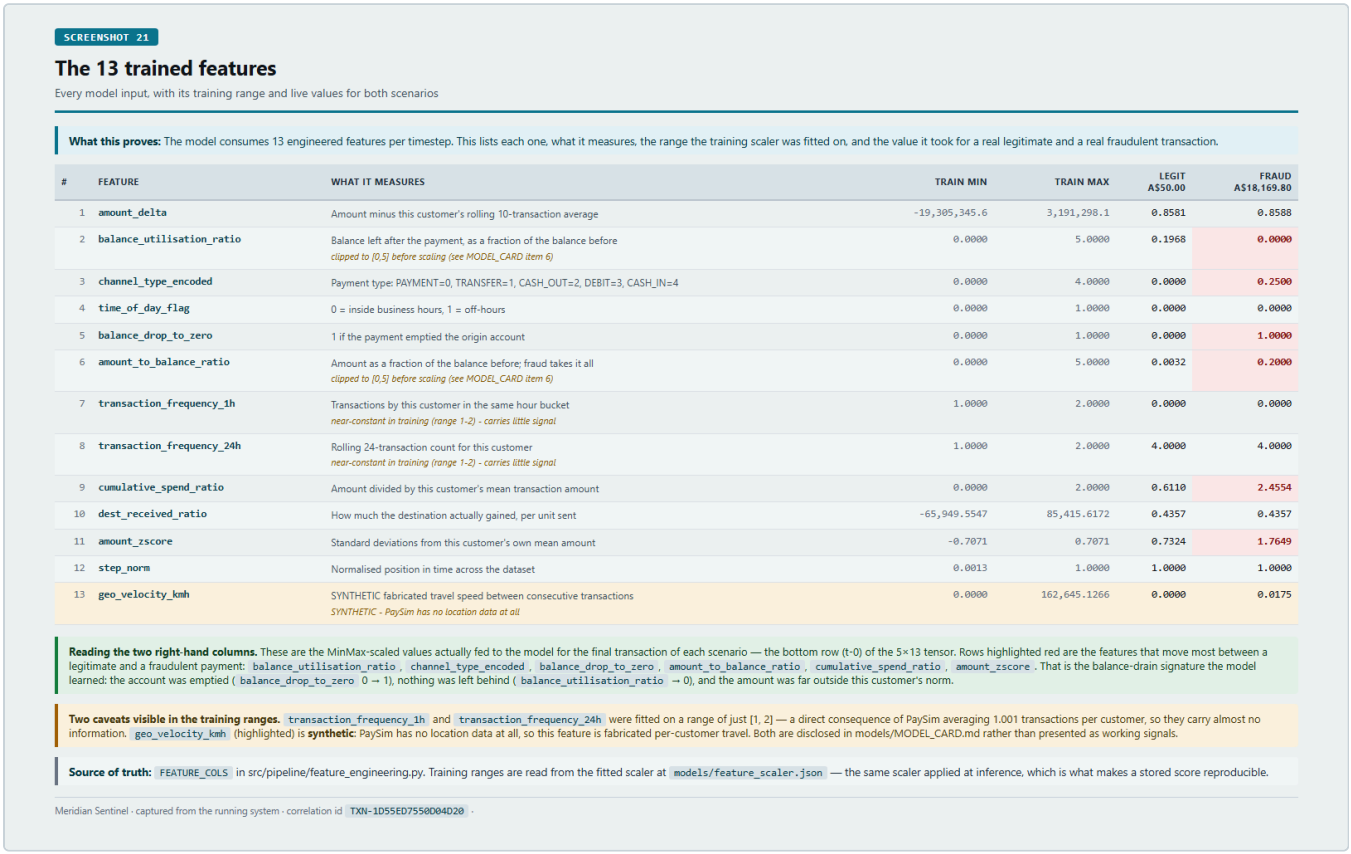
The temptation is to fit a fresh scaler on whatever batch you're currently scoring. That runs without error and returns plausible-looking numbers, but the range comes from that batch's own extremes rather than the range the network learned, so an identical transaction scores differently depending on what it happened to be sent alongside. The answers come back confidently wrong rather than obviously broken, which is the worst failure mode to debug.

So training writes its fitted scaler to `models/feature_scaler.json`, and inference loads that same file every time. Two of the ratio features are also clipped to `[0, 5]` before scaling, because both divide by the origin balance and a zero-balance account — common in PaySim — otherwise produces an enormous outlier that stretches the fitted range and squashes every real value toward zero. Clipping bounds them by construction, so training and serving see the same range without a separate threshold to keep in sync.

> **WHY THIS MATTERS FOR THE AUDIT TRAIL**
>
> Because the same scaler is used at both ends, the same transaction always produces the same tensor and therefore the same score. That reproducibility is what makes a stored score defensible — an examiner can re-run a decision and get the number that's in the record.

The 13 features

Twelve are engineered directly from the PaySim columns — amounts, balances on both sides of the transfer, transaction type and timing. The thirteenth, `geo_velocity_kmh`, is **synthesised**, because PaySim carries no location columns and the model needed a location signal. It is constructed from an assigned per-customer home city and biased toward balance-draining transfers, deliberately using observable transaction properties rather than the fraud label itself — a feature derived from the label cannot be reproduced at serving time, when the label is what the model is predicting. The construction is documented in `synthesize_geo_velocity()` and in the model card.



SCREENSHOT 21

The 13 trained features

Every model input, with its training range and live values for both scenarios

What this proves: The model consumes 13 engineered features per timestep. This lists each one, what it measures, the range the training scaler was fitted on, and the value it took for a real legitimate and a real fraudulent transaction.

#	FEATURE	WHAT IT MEASURES	TRAIN MIN	TRAIN MAX	LEGIT A$50.00	FRAUD A$18,169.80
1	amount_delta	Amount minus this customer's rolling 10-transaction average	-19,305,345.6	3,191,298.1	0.8581	0.8588
2	balance_utilisation_ratio	Balance left after the payment, as a fraction of the balance before *clipped to [0,5] before scaling (see MODEL_CARD item 6)*	0.0000	5.0000	0.1968	0.0000
3	channel_type_encoded	Payment type: PAYMENT=0, TRANSFER=1, CASH_OUT=2, DEBIT=3, CASH_IN=4	0.0000	4.0000	0.0000	0.2500
4	time_of_day_flag	0 = inside business hours, 1 = off-hours	0.0000	1.0000	0.0000	0.0000
5	balance_drop_to_zero	1 if the payment emptied the origin account	0.0000	1.0000	0.0000	1.0000
6	amount_to_balance_ratio	Amount as a fraction of the balance before; fraud takes it all *clipped to [0,5] before scaling (see MODEL_CARD item 6)*	0.0000	5.0000	0.0032	0.2000
7	transaction_frequency_1h	Transactions by this customer in the same hour bucket *near-constant in training (range 1-2) - carries little signal*	1.0000	2.0000	0.0000	0.0000
8	transaction_frequency_24h	Rolling 24-transaction count for this customer *near-constant in training (range 1-2) - carries little signal*	1.0000	2.0000	4.0000	4.0000
9	cumulative_spend_ratio	Amount divided by this customer's mean transaction amount	0.0000	2.0000	0.6110	2.4554
10	dest_received_ratio	How much the destination actually gained, per unit sent	-65,949.5547	85,415.6172	0.4357	0.4357
11	amount_zscore	Standard deviations from this customer's own mean amount	-0.7071	0.7071	0.7324	1.7649
12	step_norm	Normalised position in time across the dataset	0.0013	1.0000	1.0000	1.0000
13	geo_velocity_kmh	SYNTHETIC fabricated travel speed between consecutive transactions *SYNTHETIC - PaySim has no location data at all*	0.0000	162,645.1266	0.0000	0.0175

Reading the two right-hand columns. These are the MinMax-scaled values actually fed to the model for the final transaction of each scenario — the bottom row (t-0) of the 5×13 tensor. Rows highlighted red are the features that move most between a legitimate and a fraudulent payment: `balance_utilisation_ratio`, `channel_type_encoded`, `balance_drop_to_zero`, `amount_to_balance_ratio`, `cumulative_spend_ratio`, `amount_zscore`. That is the balance-drain signature the model learned: the account was emptied (`balance_drop_to_zero` 0 → 1), nothing was left behind (`balance_utilisation_ratio` → 0), and the amount was far outside this customer's norm.

Two caveats visible in the training ranges. `transaction_frequency_1h` and `transaction_frequency_24h` were fitted on a range of just [1, 2] — a direct consequence of PaySim averaging 1.001 transactions per customer, so they carry almost no information. `geo_velocity_kmh` (highlighted) is **synthetic**: PaySim has no location data at all, so this feature is fabricated per-customer travel. Both are disclosed in models/MODEL_CARD.md rather than presented as working signals.

Source of truth: `FEATURE_COLS` in src/pipeline/feature_engineering.py. Training ranges are read from the fitted scaler at `models/feature_scaler.json` — the same scaler applied at inference, which is what makes a stored score reproducible.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

Figure 1. Every feature with its training range and the value it took for a real legitimate and a real fraudulent transaction. Red rows are the ones that actually separate the two cases. Note rows 7 and 8 — their training range is [1, 2], which means they carry almost no information (explained in section 6).

3. Where behavioural history comes from

Both detectors need more than the payment in front of them. The model scores a five-transaction window, and Rule 5 measures a burst against the balance the customer held when the window opened. Something has to supply that history, and the choice of *what* supplies it turned out to matter more than I expected.

The memory-cache option

My original architecture put a Redis cache in this position, holding each customer's last five transactions in memory so the feature builder could read them in under a millisecond instead of querying the datastore every time. On latency alone it is the better design.

> DESIGN DECISION
>
> **Redis memory cache — proposed architecture component, not used in the final implementation.**
>
> Excluded to avoid cached or stale behavioural context affecting the consistency and reproducibility of LSTM inference. The final implementation obtains the required behavioural history directly from the authoritative transaction data source.

The reasoning is the same one behind reusing the training scaler. Those five transactions do not sit beside the model input — they *are* the model input. If the cache is even slightly behind the authoritative data, whether through a write that has not landed, an eviction under memory pressure, or a stale entry, then the same transaction scored twice produces two different tensors and therefore two different scores, with nothing in the record to indicate which one was correct.

For a system whose main claim is an audit trail an examiner can reconstruct, a score that cannot be reproduced is worth less than a score that took longer to compute. So the cache came out.

What replaced it

`attach_history()` walks a customer's transactions in strict time order and gives each one only the transactions that genuinely preceded it — no cache, and no lookahead onto payments that had not happened yet at that moment. The window handed to the model is then rebuilt from that authoritative data by `build_windows()`, scaled with the stored training scaler. The same transaction therefore always produces the same tensor, and the same score.

The same design keeps the rule engine stateless. Rule 5 needs a customer's recent run, but the correlator holds no state of its own — the caller passes the history in, exactly as it passes Rule 2 the previous transaction's coordinates. Every rule remains a pure function of its inputs, which is what makes a rule verdict reproducible from the stored evidence alone.

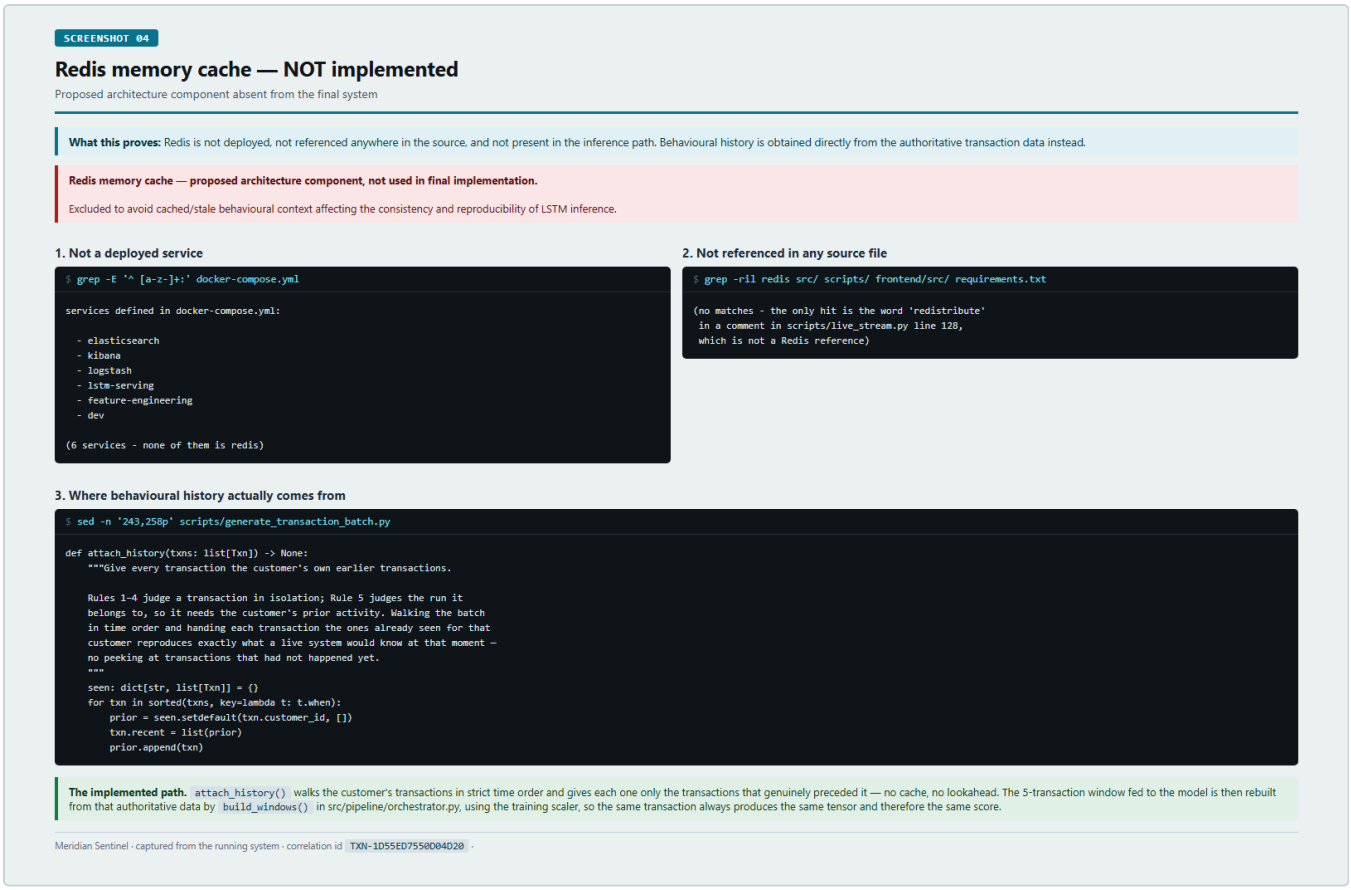


Figure 2. The implemented path. Redis is not a service in `docker-compose.yml` and is not referenced in the source; underneath is `attach_history()`, the code that actually supplies each transaction with the history that preceded it.

4. The rules

Five rules, all deterministic. Rules 1–4 look at one transaction on its own. Rule 5 is the odd one out — it reads the customer's recent run, which is what lets it catch something the others structurally can't.

RULE	CONDITION	SEVERITY
RULE_001	Amount over $10,000	HIGH
RULE_002	Geo-velocity above 500 km/h between consecutive transactions	HIGH
RULE_003	Outside 08:00–22:00 Sydney time	MEDIUM
RULE_004	Merchant on the watchlist	HIGH
RULE_005	5+ transactions in 120 min taking 20%+ of the balance	MEDIUM

The rule score is normalised by how many fired: none is 0.00, one is 0.33, two is 0.67, three or more is 1.00. That caps the rules at 0.40 of the final score, which matters — even if all five fire, the rules alone give 1.00 × 0.40 = 0.40, nowhere near the 0.70 line. Rules can't lock an account by themselves. That's deliberate.

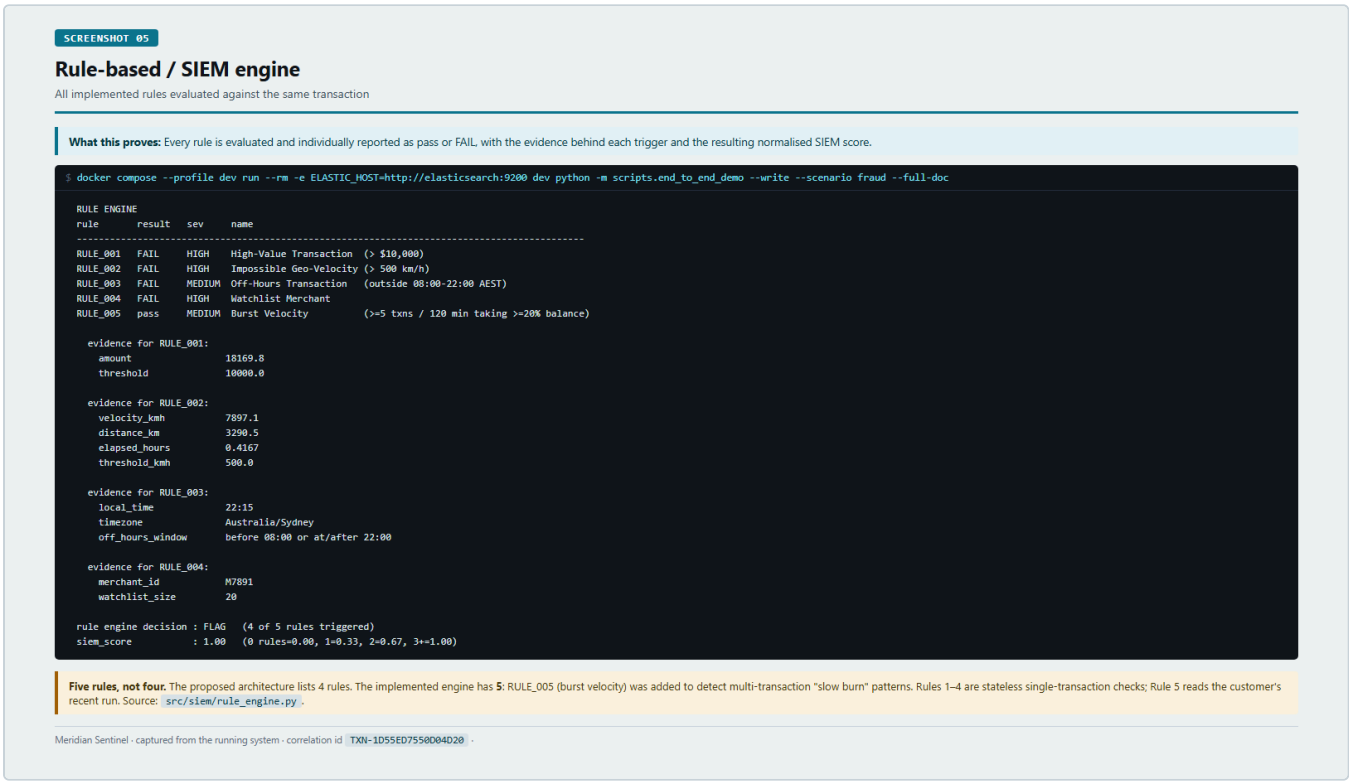


Figure 3. All five rules evaluated against one transaction, with the evidence behind each one that fired. Four fail here, giving a rule score of 1.00.

Why Rule 5 exists

Rules 1–4 share a blind spot. Someone making six small purchases over an afternoon passes every single-transaction check — each payment is small, in business hours, in one city, at a merchant nobody has watchlisted. Together they can still drain a fifth of the account. Rules 1–4 can't see that, because each one only ever looks at a single row.

Rule 5 is the only rule that reads more than the current transaction. The correlator itself stays stateless — the caller passes in the customer's recent run, the same way Rule 2 is handed the previous transaction's coordinates — so the rule engine holds no state of its own but can still reason about a sequence.

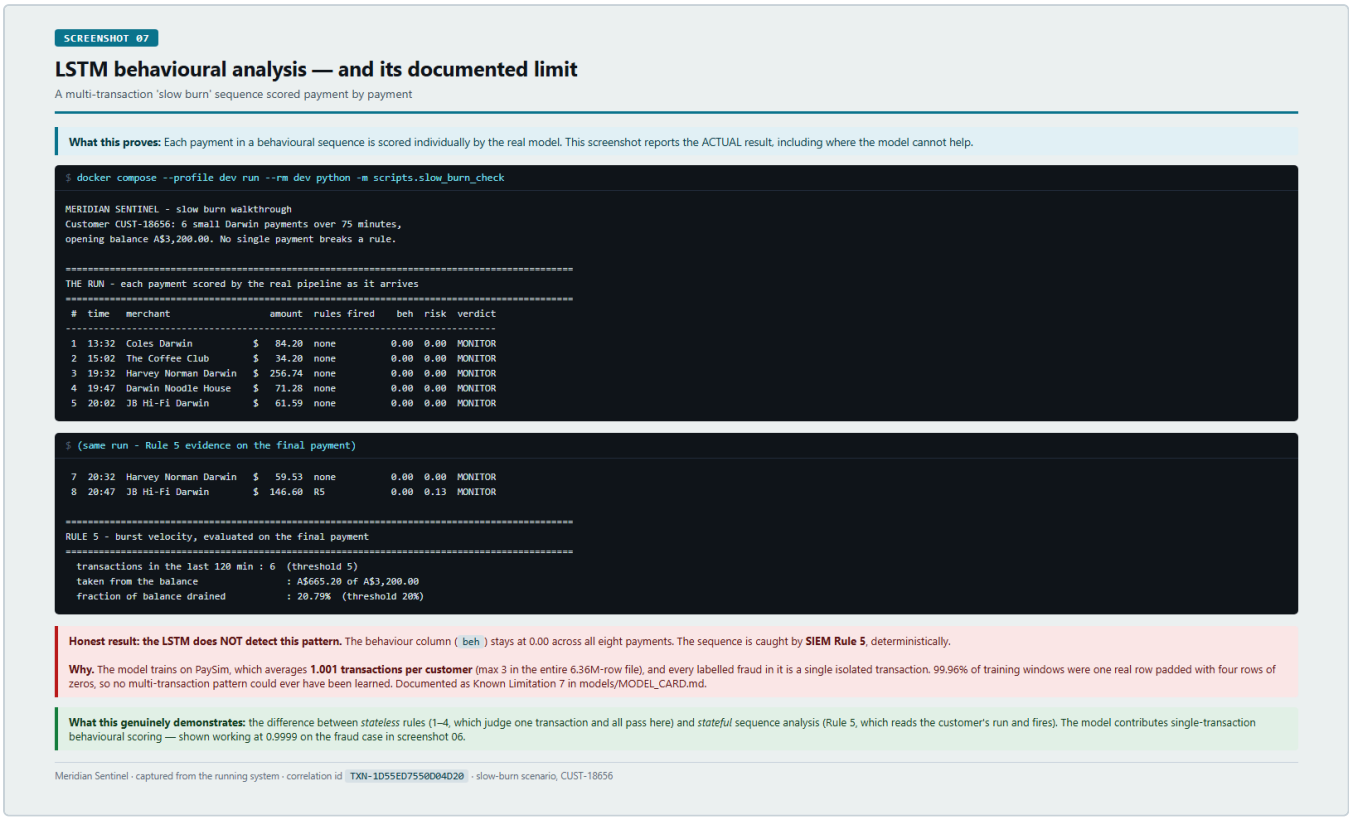


Figure 4. Rule 5 in action. Eight payments scored as they arrive — rules 1–4 pass on every single one. On the sixth payment Rule 5 fires: six transactions inside the 120-minute window taking 20.79% of the opening balance.

Note the outcome is MONITOR, not FLAGGED. One rule firing gives a rule score of 0.33, which contributes 0.132 to the blended total — well below the line. That is the intended behaviour: locking an account because somebody made six purchases in an afternoon would be a bad false positive. Rule 5's job is to put the pattern in front of an analyst with its evidence attached, not to freeze the account by itself.

5. Running it — the Docker stack

The whole system is containerised, so bringing it up is one command. Everything except the React dashboard runs as a Docker Compose service, and the dashboard is a Vite dev server I start separately.

Start everything

```
cd Meridaian
docker compose up -d
```

Then wait for the healthchecks. This matters more than it sounds — Elasticsearch and the model server both declare healthchecks, and the dev container depends on them being *healthy* rather than merely started. On a cold start Elasticsearch and the model server take about 50 seconds; Kibana needs roughly 40 seconds more before its UI will load.

```
docker compose ps
```

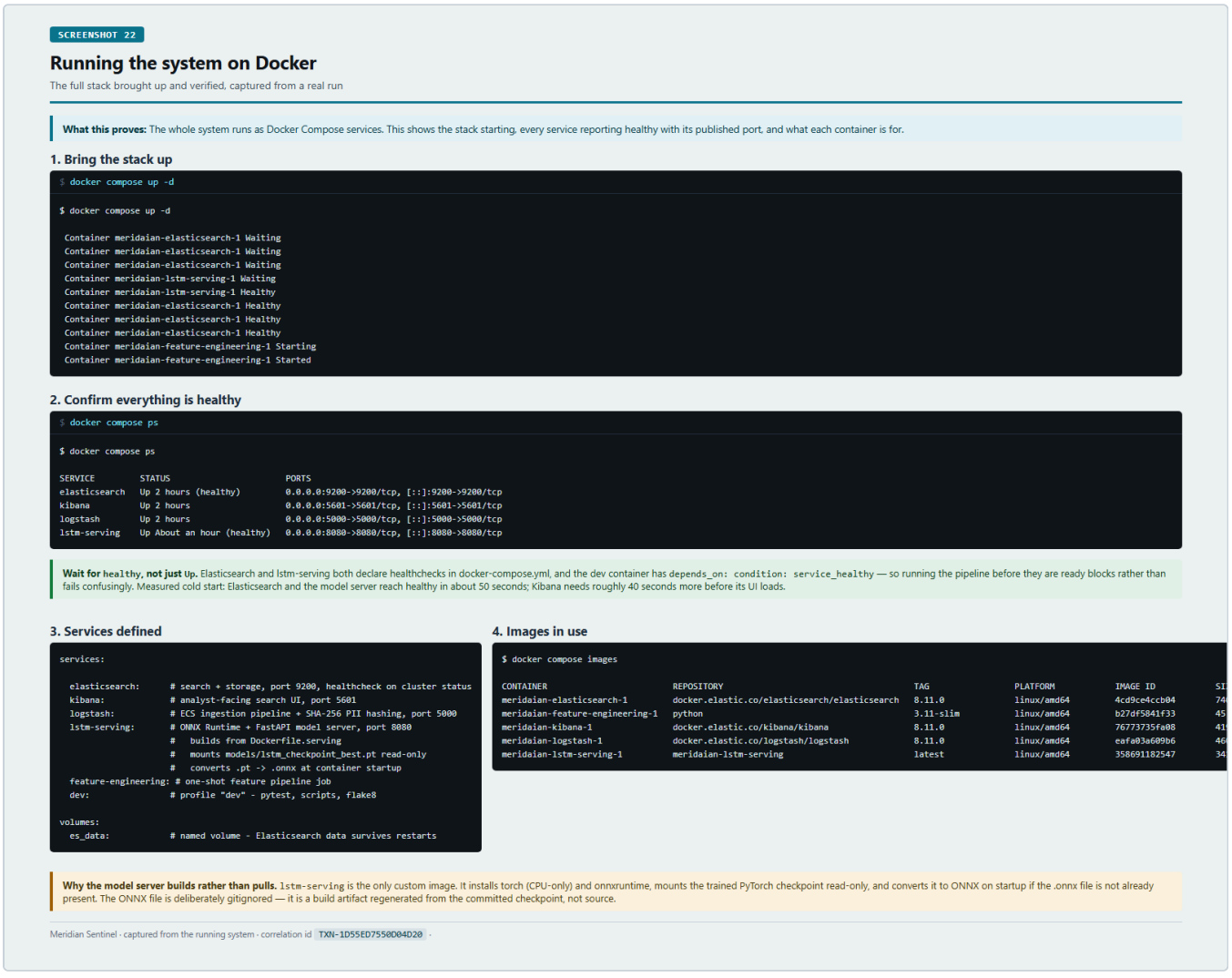


Figure 5. The stack starting and every service reporting healthy with its published port, plus what each container is for and which images they run.

The services

SERVICE	PORT	WHAT IT DOES
`elasticsearch`	9200	Transactions, incidents, notifications, audit trail
`kibana`	5601	Search UI over those indices
`logstash`	5000	ECS ingestion pipeline with SHA-256 PII hashing
`lstm-serving`	8080	ONNX Runtime + FastAPI model server
`feature-engineering`	–	One-shot feature pipeline job
`dev`	–	Profile `dev` — runs the pipeline, tests and scripts

`lstm-serving` is the only image I build rather than pull. It installs a CPU-only torch plus onnxruntime, mounts the trained checkpoint read-only, and converts the PyTorch `.pt` to ONNX on startup if the `.onnx` file isn't already there. The ONNX file is gitignored deliberately — it's a build artifact regenerated from the committed checkpoint, not source.

Start the dashboard

```
cd frontend
npm run dev    # http://localhost:5173
```

Run a transaction through

Everything runs inside the dev container so it picks up the project's dependencies and can reach the other services by their Compose names:

```
docker compose --profile dev run --rm \
    -e ELASTIC_HOST=http://elasticsearch:9200 \
    dev python -m scripts.end_to_end_demo --write
```

> **TWO THINGS THAT CATCH PEOPLE OUT**
>
> **The `ELASTIC_HOST` override is required.** Inside the dev container, `localhost` is the container itself, not Elasticsearch. Compose puts every service on one network where they resolve each other by service name, so from inside it's `http://elasticsearch:9200`. Drop that flag and the write fails.
>
> **Those hostnames only work between containers.** The pipeline logs `http://lstm-serving:8080` because that's genuinely the endpoint it called, but a browser on the host can't resolve it. From the host you use the published ports — `localhost:8080`, `localhost:9200`, `localhost:5601`.

Tests and shutdown

```
docker compose --profile dev run --rm dev pytest tests/ -v
```

```
docker compose down
```

`docker compose down` stops the containers but leaves the named volume `es_data` alone, so indexed transactions survive a restart. Adding `-v` would wipe it, which is why I don't.

6. Following one transaction all the way through

This is the part that proves the system is joined up rather than five components that happen to sit in the same repository. I gave every transaction a correlation ID at the moment it arrives, and that ID is stamped on every record after it.

The transaction: **CUST-24417**, A$18,169.80, a TRANSFER that empties the account, to a watchlisted merchant in Perth when the previous transaction was in Sydney minutes earlier, at 22:15.

Stage 1 — rules

Four of five fire. Rule score 1.00. On its own that only contributes 0.40.

Stage 2 — model

The five-transaction window is built, scaled with the training scaler, and sent to the model server as a `[1, 5, 13]` tensor. It comes back at **0.9999**.

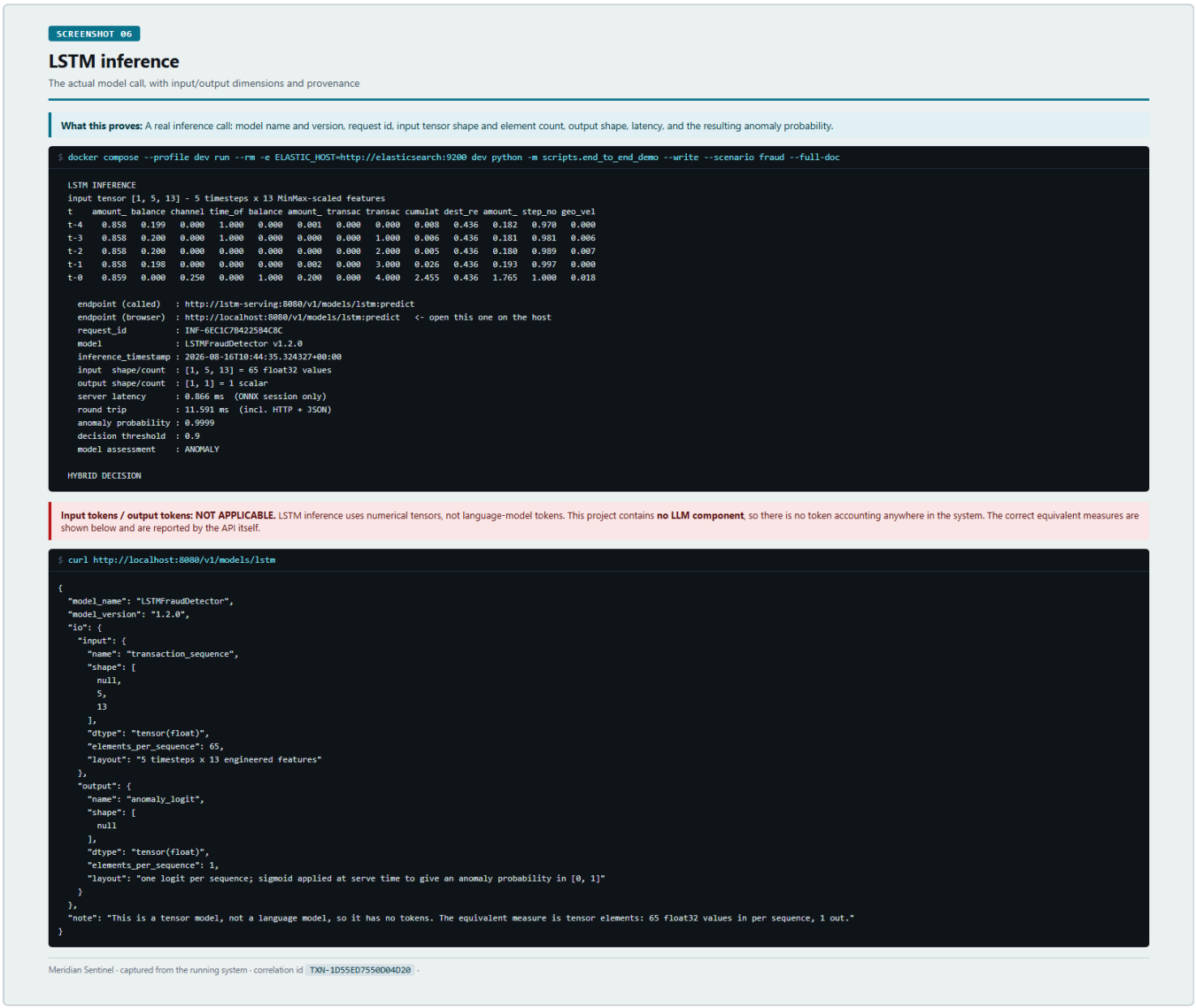


Figure 6. The actual inference call — the input tensor row by row, the request ID, model version, latency, and the score. Note the block explaining that "input/output tokens" don't apply here (section 7).

Stage 3 — blend and decide

0.9999 × 0.60 + 1.00 × 0.40 = **0.9999**. Over 0.70, so FLAGGED. The playbook fires for real: an incident record with a UUID, severity CRITICAL, action LOCK_ACCOUNT, written to Elasticsearch along with an analyst notification.

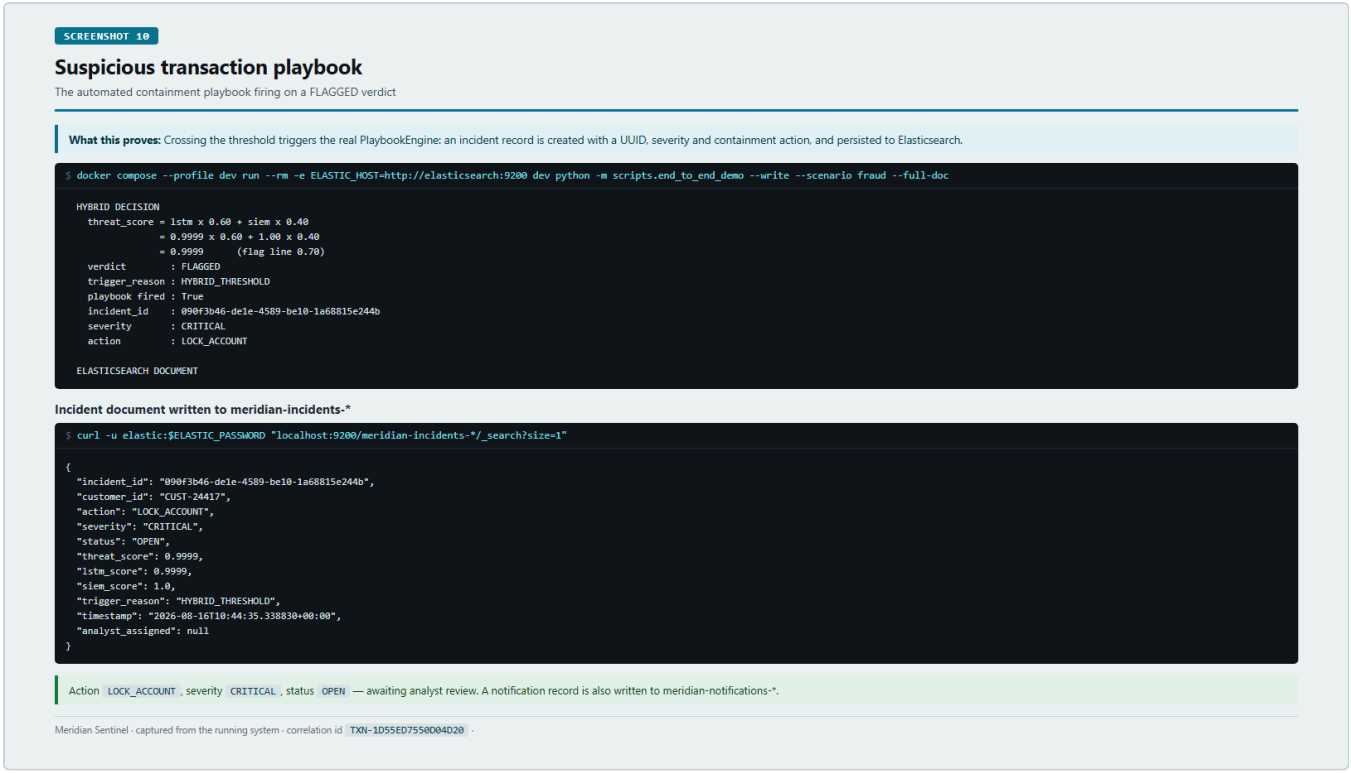


Figure 7. The playbook firing and the incident document it wrote.

Stage 4 — the analyst sees it

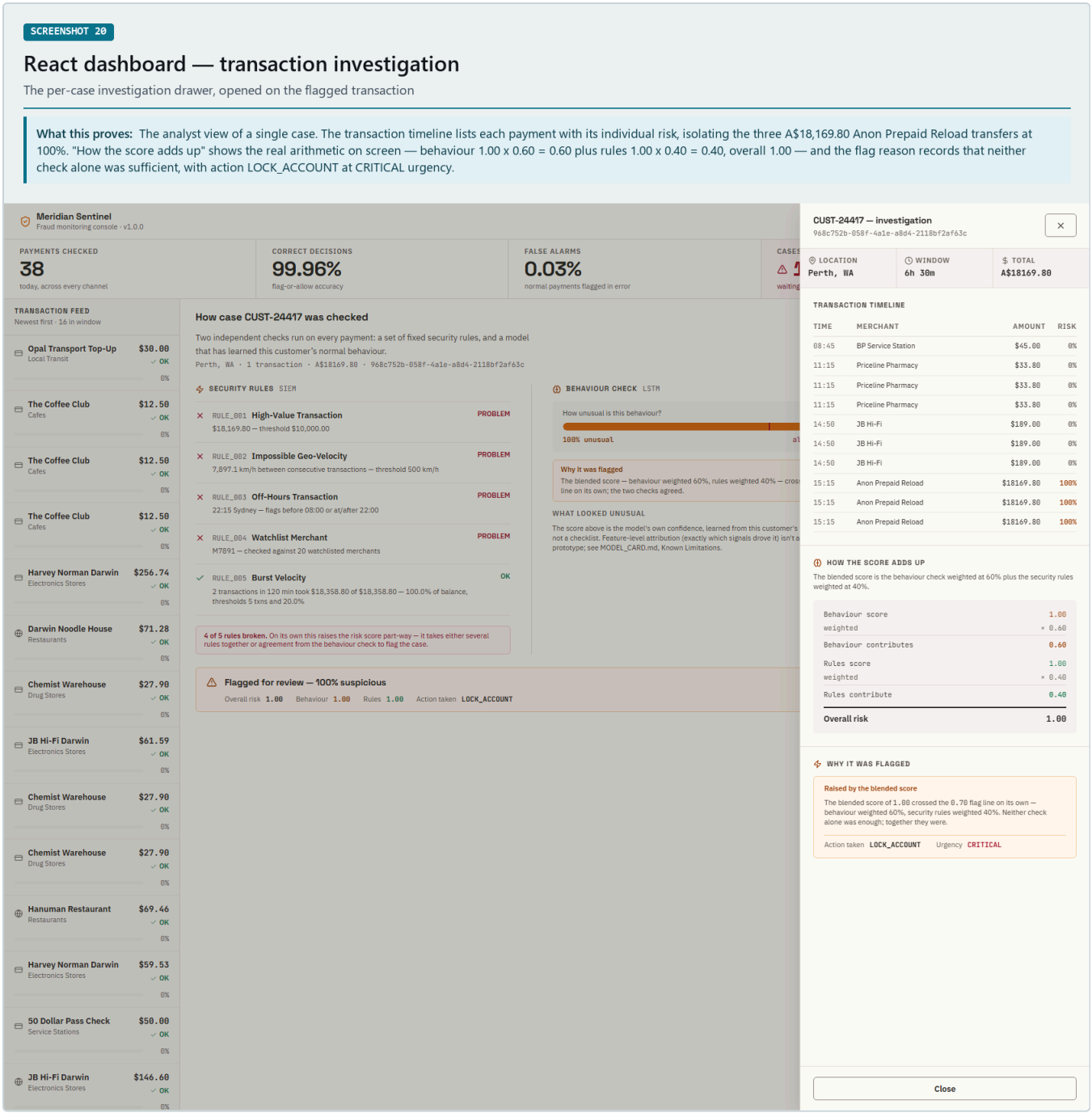


Figure 8. The case drawer in the React console. The score arithmetic is shown on screen rather than hidden — behaviour 1.00 × 0.60 plus rules 1.00 × 0.40 — along with the transaction timeline and the action taken.

Stage 5 — the audit trail

Eight stages recorded against that one correlation ID. This is the bit that makes the whole thing defensible: given an ID, you can reconstruct every decision, including which model version produced the score and how long each step took.

SCREENSHOT 19

Kibana Discover — the audit trail

All eight pipeline stages for one correlation id, Meridian Audit Trail data view

What this proves: 8 hits: the complete life of one transaction, readable in order — TRANSACTION_RECEIVED, RULES_EVALUATED (5 evaluated, 4 triggered), INFERENCE_REQUESTED, INFERENCE_RECEIVED (LSTMFraudDetector:1.2.0, 65 input elements), DECISION_GENERATED (formula 0.9999 x 0.60 + 1.0 x 0.40), PLAYBOOK_FIRED (LOCK_ACCOUNT, CRITICAL), ELASTICSEARCH_INDEXED and DASHBOARD_UPDATED. This satisfies the APRA CPS 234 and PCI DSS Requirement 10 reconstruction requirement.

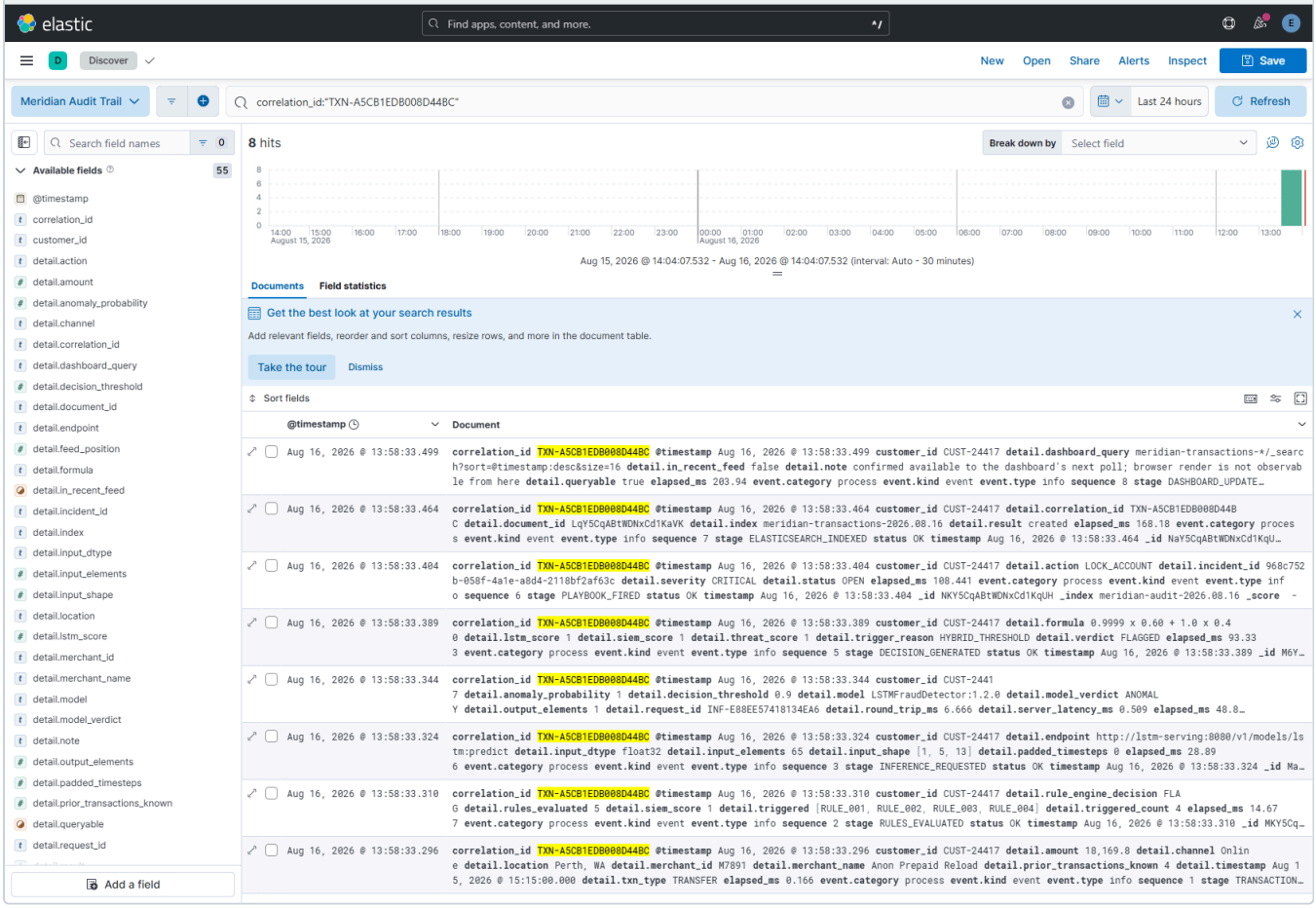


Figure 9. All eight stages in Kibana, filtered to one correlation ID — received, rules evaluated, inference requested, inference received, decision generated, playbook fired, indexed, dashboard updated.

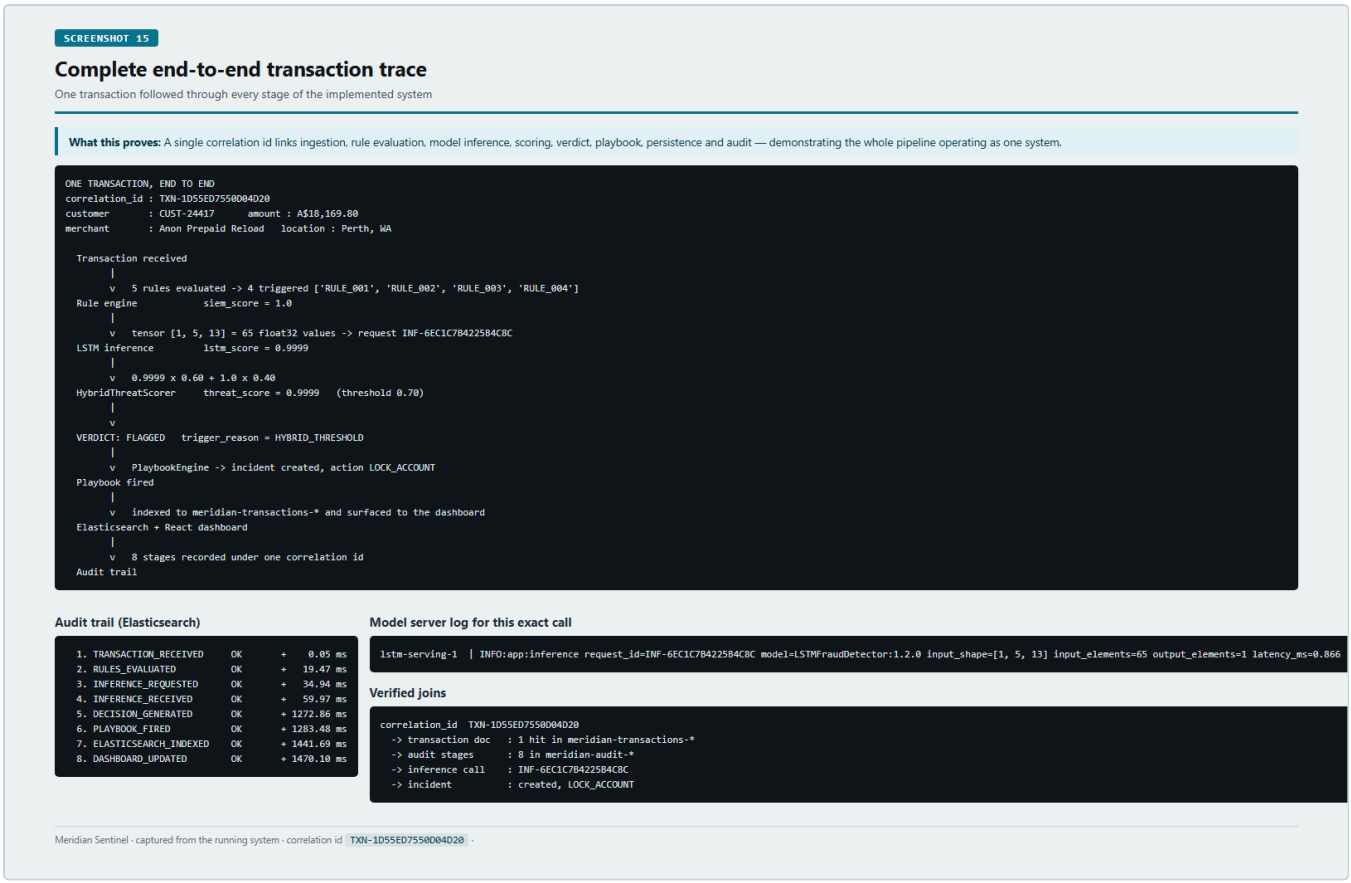


Figure 10. The same journey as one diagram, with the real values.

And the same thing for a legitimate payment

SCREENSHOT 11

SAFE (legitimate) transaction

An ordinary payment processed with no incident raised

What this proves: A legitimate transaction passes every rule, is scored as normal behaviour by the model, produces a threat score below the threshold, and fires no playbook.

```
$ docker compose --profile dev run --rm -e ELASTIC_HOST=http://elasticsearch:9200 dev python -m scripts.end_to_end_demo --write --scenario legit --full-doc

================================================================================
SCENARIO: A - LEGITIMATE  ($50.00, CUST-52210)
================================================================================
  An ordinary card payment in business hours at a clean merchant, with
  the balance left largely intact. Expect: rules ALLOW, model NORMAL,
  verdict MONITOR, no playbook.

--- transaction 1 of 5 ---------------------------------------------------------
  customer      : CUST-52210
  amount        : A$84.20
  merchant      : M0101  Coles Supermarkets  (MCC 5411 Grocery Stores)
  timestamp     : 2026-08-16 14:44:50 AEST
  location      : Sydney, NSW  (previous: Sydney, NSW)
  channel/type  : Card / PAYMENT
  balance       : A$3,240.00 -> A$3,155.80
  history known : 0 prior transaction(s) for this customer

  (context transaction — indexed for history, not individually audited; see module docstring)
```

```
$ (same run - rule engine and decision)

  RULE ENGINE
  rule      result   sev     name
  ------------------------------------------------------------------------------
  RULE_001  pass     HIGH    High-Value Transaction  (> $10,000)
  RULE_002  pass     HIGH    Impossible Geo-Velocity (> 500 km/h)
  RULE_003  pass     MEDIUM  Off-Hours Transaction   (outside 08:00-22:00 AEST)
  RULE_004  pass     HIGH    Watchlist Merchant
  RULE_005  pass     MEDIUM  Burst Velocity          (>=5 txns / 120 min taking >=20% balance)

  rule engine decision : ALLOW   (0 of 5 rules triggered)
  siem_score           : 0.00   (0 rules=0.00, 1=0.33, 2=0.67, 3+=1.00)


    playbook fired : False

  ELASTICSEARCH DOCUMENT
    _id            : xaYsCqABtWDNxCd1rqS8
{
    "@timestamp": "2026-08-16T10:44:50.243445+00:00",
    "customer_id": "CUST-52210",
    "amount": 50.0,
    "merchant_id": "M0105",
    "merchant_name": "50 Dollar Pass Check",
    "merchant_category_code": 5541,
    "mcc_label": "Service Stations",
```

Verdict MONITOR, no playbook fired, no incident created. Correlation id `TXN-91406653E26545C2` .

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` · legit transaction TXN-91406653E26545C2

Figure 11. A$50.00 at a service station. Every rule passes, the model scores 0.0000, blended score 0.0000, verdict MONITOR, no incident created.

7. Model inputs and outputs

The model consumes a fixed numerical tensor and returns a single number, so the measure of what crosses the inference boundary is the element count of those tensors. Both are instrumented and reported by the API on every call:

	NAME	SHAPE	ELEMENTS
Input	`transaction_sequence`	[1, 5, 13]	`65 float32`
Output	`anomaly_logit`	[1, 1]	`1 scalar`

Those names and shapes are read from the loaded ONNX graph at runtime rather than hardcoded, so they can't drift away from whatever model is actually being served.

SCREENSHOT 16

Model status and inference log

http://lstm-serving:8080/v1/models/lstm?limit=20&pretty

What this proves: The served model reports its own identity and tensor geometry, read from the loaded ONNX graph: input transaction_sequence [null,5,13] = 65 float32 elements per sequence, output anomaly_logit = 1 scalar. The note states plainly that this is a tensor model with no tokens. The inference_log below records every call this server has served, with request ids, element counts, latency and per-sequence assessments.


```
{
  "model_version_status": [
    {
      "version": "1",
      "state": "AVAILABLE",
      "threshold": 0.9
    }
  ],
  "model_name": "LSTMFraudDetector",
  "model_version": "1.2.0",
  "input_shape": [
    null,
    5,
    13
  ],
  "input_elements_per_sequence": 65,
  "io": {
    "input": {
      "name": "transaction_sequence",
      "shape": [
        null,
        5,
        13
      ],
      "dtype": "tensor(float)",
      "elements_per_sequence": 65,
      "layout": "5 timesteps x 13 engineered features"
    },
    "output": {
      "name": "anomaly_logit",
      "shape": [
        null
      ],
      "dtype": "tensor(float)",
      "elements_per_sequence": 1,
      "layout": "one logit per sequence; sigmoid applied at serve time to give an anomaly probability in [0, 1]"
    },
    "note": "This is a tensor model, not a language model, so it has no tokens. The equivalent measure is tensor elements: 65 float32 values in per sequence, 1 out."
  },
  "inference_log": {
    "totals": {
      "requests_served": 7,
      "sequences_scored": 14,
      "input_elements": 910,
      "output_elements": 14,
      "started_at": "2026-08-16T10:58:10.772575+00:00"
    },
    "capacity": 200,
    "returned": 7,
    "available": 7,
    "recent": [
      {
        "request_id": "INF-70796BB073AC469E",
        "timestamp": "2026-08-16T10:58:42.140729+00:00",
        "input_shape": [
          1,
          5,
          13
        ],
        "input_elements": 65,
        "output_shape": [
          1,
          1
        ],
        "output_elements": 1,
        "latency_ms": 0.261,
        "anomaly_probabilities": [
          0.0
        ],
        "assessment": [
          "NORMAL"
        ]
      },
      {
        "request_id": "INF-8E3F52A4437A487F",
        "timestamp": "2026-08-16T10:58:42.110500+00:00",
        "input_shape": [
          8,
          5,
          13
        ],
        "input_elements": 520,
        "output_shape": [
          8,
          1
        ],
        "output_elements": 8,
        "latency_ms": 0.612,
        "anomaly_probabilities": [
          0.0,
          0.0919,
          0.0,
          0.0021,
          0.0,
          0.0,
          0.0,
          0.0002
        ],
        "assessment": [
          "NORMAL",
          "NORMAL",
          "NORMAL",
          "NORMAL",
          "NORMAL",
          "NORMAL",
          "NORMAL",
          "NORMAL"
        ]
      },
      {
        "request_id": "INF-6376AE1D0A594566",
        "timestamp": "2026-08-16T10:58:42.090392+00:00",
        "input_shape": [
          1,
          5,
          13
        ],
        "input_elements": 65,
        "output_shape": [
          1,
          1
        ],
        "output_elements": 1,
        "latency_ms": 0.273,
```

Figure 12. The model status endpoint reporting its own tensor geometry, and a rolling log of every inference it has served with element counts and latency.

8. Storage and the dashboard

Every transaction is written to Elasticsearch carrying the same facts in two shapes within one document: flat fields (`amount`, `merchant_name`, `lstm_score`) that the React dashboard reads directly, and nested ECS fields (`transaction.*`, `source.geo.*`, `event.*`) that Kibana aggregates on. Two consumers, two expectations about field layout, one write — writing only one shape leaves the other reading an index it can't interpret.

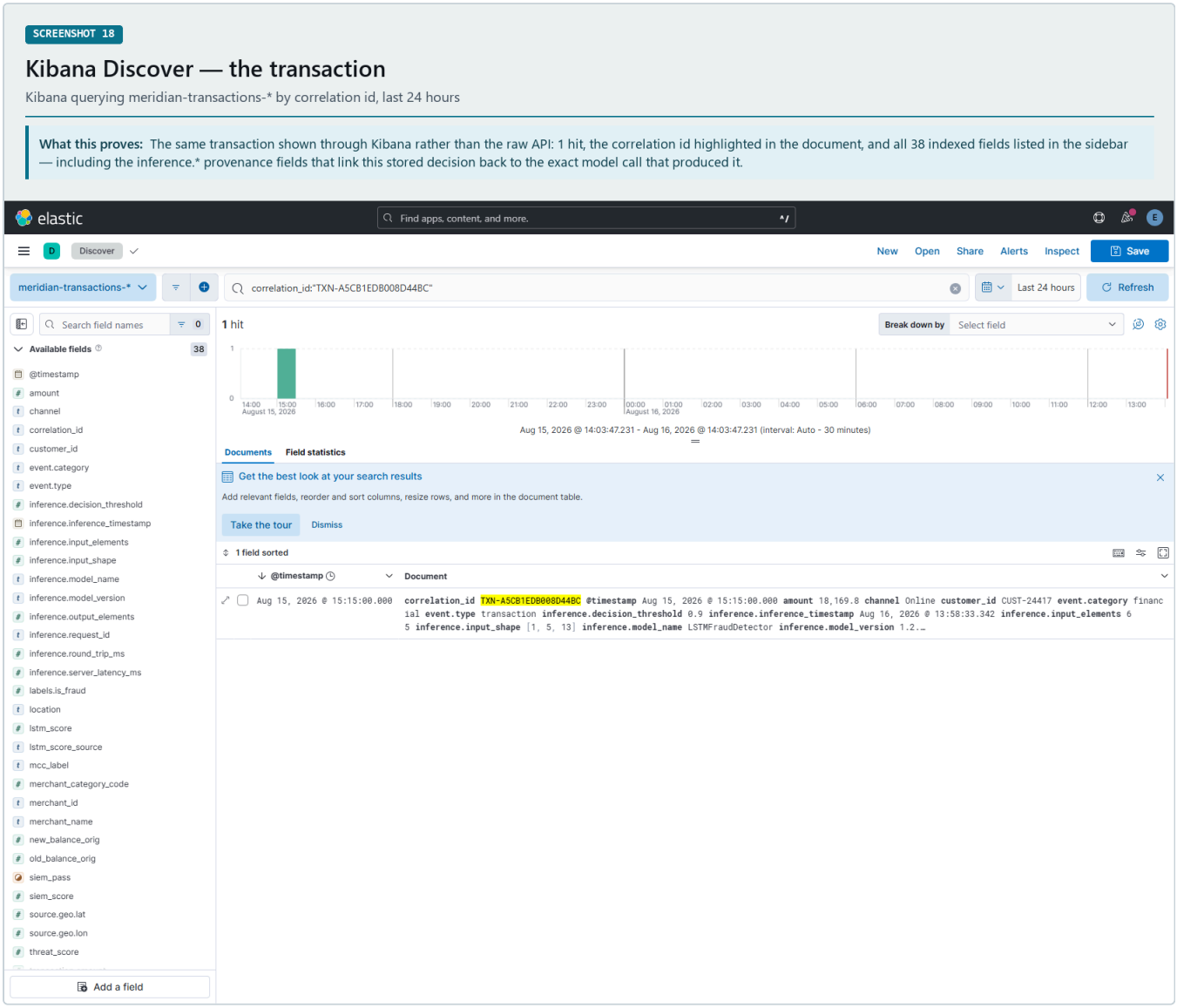


Figure 13. The transaction in Kibana, found by correlation ID. The sidebar shows all 38 indexed fields, including the `inference.*` fields that tie the stored score back to the exact model call.

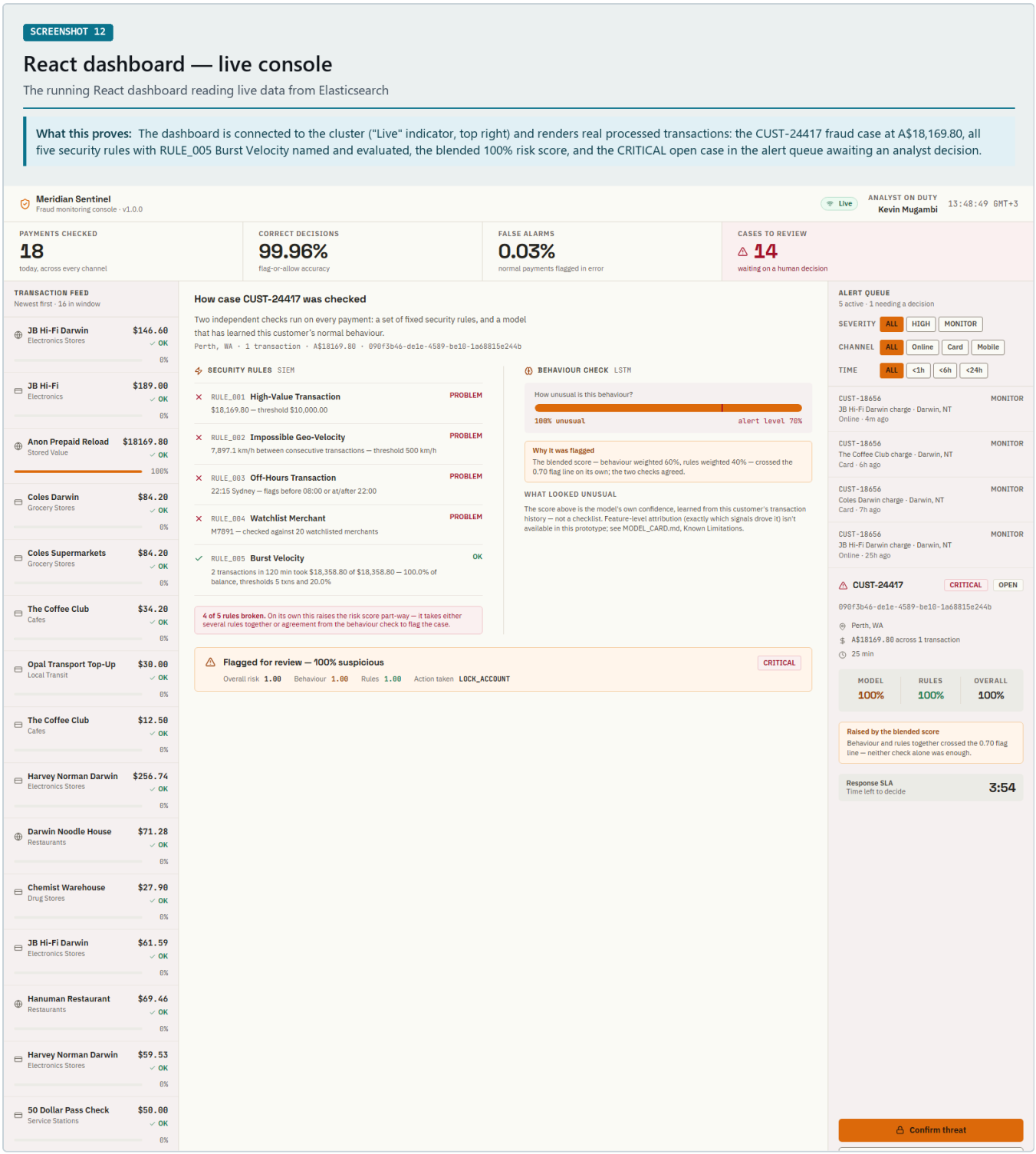


Figure 14. The dashboard reading live from Elasticsearch. I removed all the mock data during development — if the cluster is empty it shows an empty state rather than fake rows.

9. Future work

The prototype does what it set out to do. These are the directions I would take it next.

1. **Train on richer transaction histories.** The architecture already supports a five-transaction window. A dataset with longer per-customer sequences would let the model learn patterns that build across several payments, rather than scoring each one on its own merits. That is a data question rather than a code change — the pipeline is already shaped for it.
2. **Add feature-level explainability.** The console reports a score and the rules behind a decision, but not which individual features moved the model. Adding SHAP would let an analyst see the exact contribution of each of the thirteen inputs, which is the natural next step for a system where a human has to sign off on the outcome.
3. **Route the live path through Logstash.** The ECS pipeline is written, deployed and working, including SHA-256 hashing of personally identifying fields. Moving the demonstrated path onto it would exercise the ingestion tier under the same load as the rest of the system.
4. **Enforce TLS across services.** The TLS 1.3 configuration and certificate generation script are in the repository. Turning enforcement on across every service URL and healthcheck is the remaining step toward the PCI DSS network posture the design targets.
5. **Extend the rule set.** Rule 5 showed the value of a rule that reads a customer's recent run rather than one payment. The same shape could cover other multi-transaction patterns — escalating amounts, unusual merchant sequences, dormant accounts waking up.

> **REPRODUCING THIS**
>
> ```
> docker compose up -d
> docker compose --profile dev run --rm \
>     -e ELASTIC_HOST=http://elasticsearch:9200 \
>     dev python -m scripts.end_to_end_demo --write
> ```
>
> That runs all three scenarios and prints every stage. The screenshots in this report came from that command and from the running dashboard, Kibana and model server.

Appendix — full evidence set

All screenshots, captured from the running system. Each carries its own explanation. The figures above are a selection; these are the complete set in order.

#	SCREENSHOT	SHOWS
01	transaction-ingestion	Transaction enters, gets a correlation ID
02	data-ingestion-normalisation	Flat + nested ECS document shapes
03	feature-engineering	History becomes a 5×13 scaled tensor
04	redis-not-implemented	Behavioural history read from authoritative data
05	rule-engine	All 5 rules with evidence
06	lstm-inference	The model call and its metadata
07	lstm-behaviour-analysis	Slow burn scored payment by payment
08	hybrid-threat-score	The 60/40 blend
09	final-verdict	Verdict against the threshold
10	suspicious-playbook	Incident created, account locked
11	safe-transaction	Legitimate payment, no incident
12	react-dashboard	Live console
13	elasticsearch-event	Stored document + all indices
14	audit-logs	8-stage trail
15	end-to-end-flow	Whole pipeline, one ID
16	model-status-inference-log	Tensor geometry + served calls
17	elasticsearch-browser	Document via the REST API
18	kibana-discover-transaction	Transaction in Kibana
19	kibana-audit-trail	Audit trail in Kibana
20	react-dashboard-transaction	Case investigation drawer
21	thirteen-features	Every feature with live values

SCREENSHOT 01

Transaction entering the system

A real transaction submitted into the fraud-detection pipeline

What this proves: A concrete transaction (customer, amount, location, channel, timestamp, balances) is accepted by the pipeline and assigned a correlation id that follows it to every later stage.

```
$ docker compose --profile dev run --rm -e ELASTIC_HOST=http://elasticsearch:9200 dev python -m scripts.end_to_end_demo --write --scenario fraud --full-doc

================================================================================
SCENARIO: C - FLAGGED FRAUD  (full-balance TRANSFER, CUST-24417)
================================================================================
  A balance-draining TRANSFER to a watchlisted mule account in another
  city, off-hours. Expect: several rules FAIL, model ANOMALY, verdict
  FLAGGED, and the playbook writes a real incident.

--- transaction 1 of 5 -----------------------------------------------------
  customer      : CUST-24417
  amount        : A$62.40
  merchant      : M0102  Woolworths  (MCC 5411 Grocery Stores)
  timestamp     : 2026-08-15 11:15:00 AEST
  location      : Sydney, NSW  (previous: Sydney, NSW)
  channel/type  : Card / PAYMENT
  balance       : A$18,500.00 -> A$18,437.60
  history known : 0 prior transaction(s) for this customer

  (context transaction — indexed for history, not individually audited; see module docstring)
```

The transaction is stamped with correlation id `TXN-1D55ED7550D04D20`. Screenshots 03–15 all follow this same id.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A1. Transaction entering the pipeline.

SCREENSHOT 03

Feature engineering

Raw transaction history converted into the model input tensor

What this proves: The customer's 5-transaction window is turned into a numerical tensor of 13 MinMax-scaled engineered features per timestep, which is what the LSTM receives.

```
$ docker compose --profile dev run --rm -e ELASTIC_HOST=http://elasticsearch:9200 dev python -m scripts.end_to_end_demo --write --scenario fraud --full-doc

  LSTM INFERENCE
  input tensor [1, 5, 13] - 5 timesteps x 13 MinMax-scaled features
  t    amount_ balance channel time_of balance amount_ transac transac cumulat dest_re amount_ step_no geo_vel
  t-4   0.858   0.199   0.000   1.000   0.000   0.001   0.000   0.000   0.008   0.436   0.182   0.970   0.000
  t-3   0.858   0.200   0.000   1.000   0.000   0.000   0.000   1.000   0.006   0.436   0.181   0.981   0.006
  t-2   0.858   0.200   0.000   0.000   0.000   0.000   0.000   2.000   0.005   0.436   0.180   0.989   0.007
  t-1   0.858   0.198   0.000   0.000   0.000   0.002   0.000   3.000   0.026   0.436   0.193   0.997   0.000
  t-0   0.859   0.000   0.250   0.000   1.000   0.200   0.000   4.000   2.455   0.436   1.765   1.000   0.018

    endpoint (called)   : http://lstm-serving:8080/v1/models/lstm:predict
    endpoint (browser)  : http://localhost:8080/v1/models/lstm:predict   <- open this one on the host
    request_id          : INF-6EC1C7B422584C8C
    model               : LSTMFraudDetector v1.2.0
    inference_timestamp : 2026-08-16T10:44:35.324327+00:00
    input  shape/count  : [1, 5, 13] = 65 float32 values
    output shape/count  : [1, 1] = 1 scalar
    server latency      : 0.866 ms  (ONNX session only)
    round trip          : 11.591 ms  (incl. HTTP + JSON)
    anomaly probability : 0.9999
    decision threshold  : 0.9
    model assessment    : ANOMALY
```

13 features, not 12. The proposed architecture specifies 12 features and a 5×12 input. The implemented model was retrained on **13** features — a 13th, `geo_velocity_kmh`, was added later. The real input shape is therefore **[1, 5, 13]**. The feature list is `FEATURE_COLS` in src/pipeline/feature_engineering.py. Rows marked *(zero-pad)* are timesteps before the customer had that much history.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A2. Raw history converted to the model input tensor.

SCREENSHOT 08

HybridThreatScorer

The two engine scores combined into one threat score

What this proves: The blended score is computed from real runtime values using the documented formula threat = (LSTM x 0.60) + (SIEM x 0.40), and compared against the 0.70 threshold.

```
$ docker compose --profile dev run --rm -e ELASTIC_HOST=http://elasticsearch:9200 dev python -m scripts.end_to_end_demo --write --scenario fraud --full-doc

  HYBRID DECISION
    threat_score = lstm x 0.60 + siem x 0.40
                 = 0.9999 x 0.60 + 1.00 x 0.40
                 = 0.9999       (flag line 0.70)
    verdict        : FLAGGED
    trigger_reason : HYBRID_THRESHOLD
    playbook fired : True
    incident_id    : 090f3b46-de1e-4589-be10-1a68815e244b
    severity       : CRITICAL
    action         : LOCK_ACCOUNT

  ELASTICSEARCH DOCUMENT
```

Both engines agree independently. Note that the rule engine alone could not reach the threshold: 4 rules firing gives a SIEM score of 1.00, contributing only 1.00 × 0.40 = **0.40**. Crossing 0.70 requires the behavioural model to agree.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A3. The blend, with real runtime values.

SCREENSHOT 09

Final verdict

The decision returned by the system for the fraud transaction

What this proves: The final verdict is derived from real runtime values: both engine scores, the blended threat score, the threshold it is compared against, and the trigger reason.

```
$ docker compose --profile dev run --rm -e ELASTIC_HOST=http://elasticsearch:9200 dev python -m scripts.end_to_end_demo --write --scenario fraud --full-doc

  HYBRID DECISION
    threat_score = lstm x 0.60 + siem x 0.40
                 = 0.9999 x 0.60 + 1.00 x 0.40
                 = 0.9999      (flag line 0.70)
    verdict        : FLAGGED
    trigger_reason : HYBRID_THRESHOLD
    playbook fired : True
    incident_id    : 090f3b46-de1e-4589-be10-1a68815e244b
    severity       : CRITICAL
    action         : LOCK_ACCOUNT

  ELASTICSEARCH DOCUMENT
```

```
$ (the stored verdict fields on the Elasticsearch document)

{
  "correlation_id": "TXN-1D55ED7550D04D20",
  "customer_id": "CUST-24417",
  "amount": 18169.8,
  "lstm_score": 0.9999,
  "siem_score": 1.0,
  "threat_score": 0.9999,
  "verdict": "FLAGGED",
  "trigger_reason": "HYBRID_THRESHOLD",
  "triggered_rules": [
    "RULE_001",
    "RULE_002",
    "RULE_003",
    "RULE_004"
  ],
  "siem_pass": false,
  "lstm_score_source": "model"
}
```

THREAT SCORE: 0.9999 THRESHOLD: 0.70 VERDICT: FLAGGED

This system labels the two outcomes `FLAGGED` and `MONITOR` rather than SUSPICIOUS/SAFE. MONITOR is not "ignored" — the transaction is logged, scored and surfaced to the analyst queue; it simply does not trigger automated containment.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A4. Final verdict against the 0.70 threshold.

SCREENSHOT 14

Audit logs

The complete stage-by-stage audit trail, retrieved from Elasticsearch

What this proves: Every stage of the decision is recorded against one correlation id, in order, with timestamps and elapsed timings — satisfying APRA CPS 234 and PCI DSS Req. 10 reconstruction requirements.

```
$ curl -u elastic:$ELASTIC_PASSWORD "localhost:9200/meridian-audit-*/_search?q=correlation_id.keyword:%22TXN-1D55ED7550D04D20%22&sort=sequence:asc"

correlation_id : TXN-1D55ED7550D04D20
customer_id    : CUST-24417
stages         : 8

 1. TRANSACTION_RECEIVED     OK       +    0.052 ms   10:44:35.267
 2. RULES_EVALUATED          OK       +   19.467 ms   10:44:35.286
 3. INFERENCE_REQUESTED      OK       +   34.944 ms   10:44:35.302
 4. INFERENCE_RECEIVED       OK       +   59.969 ms   10:44:35.327
 5. DECISION_GENERATED       OK       + 1272.855 ms   10:44:36.540
 6. PLAYBOOK_FIRED           OK       + 1283.476 ms   10:44:36.550
 7. ELASTICSEARCH_INDEXED    OK       + 1441.692 ms   10:44:36.709
 8. DASHBOARD_UPDATED        OK       + 1470.098 ms   10:44:36.737
```

```
$ cat logs/audit/meridian-audit-$(date +%F).jsonl | head -2

{
  "@timestamp": "2026-08-16T10:44:35.267582+00:00",
  "timestamp": "2026-08-16T10:44:35.267582+00:00",
  "correlation_id": "TXN-1D55ED7550D04D20",
  "customer_id": "CUST-24417",
  "stage": "TRANSACTION_RECEIVED",
  "status": "OK",
  "sequence": 1,
  "elapsed_ms": 0.052,
  "detail": {
    "amount": 18169.8,
    "merchant_id": "M7891",
    "merchant_name": "Anon Prepaid Reload",
    "location": "Perth, WA",
    "channel": "Online",
    "txn_type": "TRANSFER",
    "timestamp": "2026-08-15T12:15:00+00:00",
    "prior_transactions_known": 4
  },
  "event": {
    "category": "process",
    "type": "info",
    "kind": "event"
  }
}
  ...
```

The same trail is written to two independent sinks: the `meridian-audit-*` Elasticsearch index (durable, queryable) and `logs/audit/*.jsonl` (local, needs no cluster).

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A5. The audit trail as stored, with per-stage timings.

SCREENSHOT 17

Elasticsearch — document retrieved in the browser

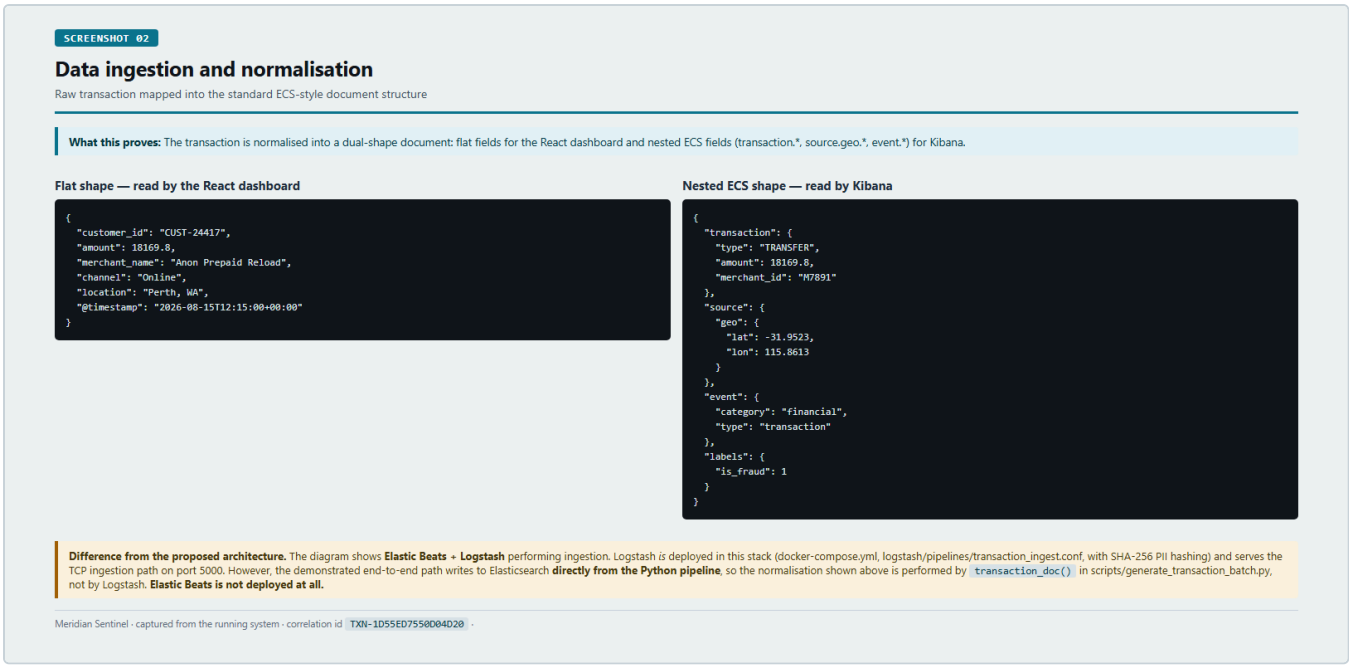
Querying the traced transaction directly from the Elasticsearch REST API

> **What this proves:** The transaction is durably stored and retrievable by its correlation id — exactly 1 hit. The document carries the full decision trail: lstm_score 0.9999, siem_score 1.0, threat_score 0.9999, verdict FLAGGED, the four triggered rules, and a nested inference block naming the model version, request id, latency and tensor element counts.

Pretty-print


```
{
  "took" : 43,
  "timed_out" : false,
  "_shards" : {
    "total" : 6,
    "successful" : 6,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 1,
      "relation" : "eq"
    },
    "max_score" : 2.302585,
    "hits" : [
      {
        "_index" : "meridian-transactions-2026.08.16",
        "_id" : "LqY5CqABtWDNxCd1KaVK",
        "_score" : 2.302585,
        "_source" : {
          "@timestamp" : "2026-08-15T12:15:00+00:00",
          "customer_id" : "CUST-24417",
          "amount" : 18169.8,
          "merchant_id" : "M7891",
          "merchant_name" : "Anon Prepaid Reload",
          "merchant_category_code" : 6540,
          "mcc_label" : "Stored Value",
          "channel" : "Online",
          "location" : "Perth, WA",
          "siem_pass" : false,
          "lstm_score" : 0.9999,
          "transaction" : {
            "type" : "TRANSFER",
            "amount" : 18169.8,
            "merchant_id" : "M7891"
          },
          "labels" : {
            "is_fraud" : 1
          },
          "source" : {
            "geo" : {
              "lat" : -31.9523,
              "lon" : 115.8613
            }
          },
          "event" : {
            "category" : "financial",
            "type" : "transaction"
          },
          "siem_score" : 1.0,
          "threat_score" : 0.9999,
          "verdict" : "FLAGGED",
          "trigger_reason" : "HYBRID_THRESHOLD",
          "triggered_rules" : [
            "RULE_001",
            "RULE_002",
            "RULE_003",
            "RULE_004"
          ],
          "old_balance_orig" : 18169.8,
          "new_balance_orig" : 0.0,
          "lstm_score_source" : "model",
          "correlation_id" : "TXN-A5CB1EDB008D44BC",
          "inference" : {
            "request_id" : "INF-E88EE57418134EA6",
            "model_name" : "LSTMFraudDetector",
            "model_version" : "1.2.0",
            "inference_timestamp" : "2026-08-16T10:58:33.342853+00:00",
            "server_latency_ms" : 0.509,
            "round_trip_ms" : 6.666,
            "input_shape" : [
              1,
              5,
              13
            ],
            "input_elements" : 65,
            "output_elements" : 1,
            "decision_threshold" : 0.9
          }
        }
      }
    ]
  }
}
```


A6. The stored document retrieved from Elasticsearch.



A7. Normalisation into both document shapes.

SCREENSHOT 13

Elasticsearch — event persisted

The same transaction stored and retrievable from the cluster

What this proves: The transaction is durably persisted in Elasticsearch with its scores, verdict, audit fields and correlation id, and is retrievable by that id.

```
$ curl -u elastic:$ELASTIC_PASSWORD "localhost:9200/meridian-transactions-*/_search?q=correlation_id.keyword:%22TXN-1D55ED7550D04D20%22"

{
  "_index": "meridian-transactions-2026.08.16",
  "_id": "uKYsCqABtWDNxCd1ZKRS",
  "hits_total": 1
}
```

Stored document


```
{
  "@timestamp": "2026-08-15T12:15:00+00:00",
  "customer_id": "CUST-24417",
  "amount": 18169.8,
  "merchant_id": "M7891",
  "merchant_name": "Anon Prepaid Reload",
  "merchant_category_code": 6540,
  "mcc_label": "Stored Value",
  "channel": "Online",
  "location": "Perth, WA",
  "siem_pass": false,
  "lstm_score": 0.9999,
  "transaction": {
    "type": "TRANSFER",
    "amount": 18169.8,
    "merchant_id": "M7891"
  },
  "labels": {
    "is_fraud": 1
  },
  "source": {
    "geo": {
      "lat": -31.9523,
      "lon": 115.8613
    }
  },
  "event": {
    "category": "financial",
    "type": "transaction"
  },
  "siem_score": 1.0,
  "threat_score": 0.9999,
  "verdict": "FLAGGED",
  "trigger_reason": "HYBRID_THRESHOLD",
  "triggered_rules": [
    "RULE_001",
    "RULE_002",
    "RULE_003",
    "RULE_004"
  ],
  "old_balance_orig": 18169.8,
  "new_balance_orig": 0.0,
  "lstm_score_source": "model",
  "correlation_id": "TXN-1D55ED7550D04D20",
  "inference": {
    "request_id": "INF-6EC1C78422584C8C",
    "model_name": "LSTMFraudDetector",
    "model_version": "1.2.0",
    "inference_timestamp": "2026-08-16T10:44:35.324327+00:00",
    "server_latency_ms": 0.866,
    "round_trip_ms": 11.591,
    "input_shape": [
      1,
      5,
      13
    ],
    "input_elements": 65,
    "output_elements": 1,
    "decision_threshold": 0.9
  }
}
```


All Meridian indices in the cluster

```
$ curl -u elastic:$ELASTIC_PASSWORD 'localhost:9200/_cat/indices/meridian-*?v'

index                              docs.count store.size
meridian-notifications-2026.08.16          1     10.3kb
meridian-incidents-2026.08.10              1     19.1kb
meridian-incidents-2026.08.14              5     72.2kb
meridian-incidents-2026.08.13              1      8kb
meridian-incidents-2026.08.16              1     21.1kb
meridian-incidents-2026.08.15              9    175.9kb
meridian-audit-2026.08.15                505     396kb
meridian-audit-2026.08.16                 71    128.3kb
meridian-transactions-2026.08.14          46    211.1kb
meridian-transactions-2026.08.13           6     27.5kb
meridian-audit-2026.08.14                158     271kb
meridian-transactions-2026.08.16          18     61.8kb
meridian-transactions-2026.08.15         131     199kb
meridian-transactions-2026.08.10           2     19.5kb
meridian-transactions-2026.06.30           3     46.7kb
meridian-notifications-2026.08.10          1     10.4kb
meridian-notifications-2026.08.15          8     72.6kb
meridian-notifications-2026.08.14          4     31.6kb
```

Why this is the Elasticsearch REST API and not a Kibana screenshot. Kibana is deployed and running (port 5601) with three data views already created — `meridian-transactions-*`, `meridian-incidents-*` and `meridian-audit-*` — but it requires an interactive browser login that cannot be automated headlessly. The query above is the same data Kibana displays, taken directly from the source. To reproduce in Kibana: open localhost:5601/app/discover, select the meridian-transactions-* data view, **set the time range to "Last 24 hours"**, and query `correlation_id : "TXN-1D55ED7550D04D20"`.

Meridian Sentinel · captured from the running system · correlation id `TXN-1D55ED7550D04D20` ·

A8. Stored document and every Meridian index in the cluster.